

SPRINT 2

TAREA 5

Agregar Tareas de manera Programada

PROYECTO INDIVIDUAL · PLANIFICADOR DE TAREAS

JavaScript · Clases · Objetos · Arrays · Formularios · Eventos

NUEVO NIVEL DE INTERACCIÓN

En esta tarea conectarás la estructura TaskManager con el formulario Nueva Tarea. La aplicación dejará de tener únicamente una interfaz y comenzará a registrar, mediante JavaScript, las tareas que el usuario crea.

Duración estimada

1 hora y 45 minutos

Tecnologías

HTML5 · Bootstrap 5 · JavaScript

OBJETIVO

Implementar el método `addTask()` en la clase `TaskManager` para registrar nuevas tareas, asignarles un identificador único y conectar este comportamiento con el formulario Nueva Tarea.

RESULTADO ESPERADO

- `TaskManager` conserva un contador `currentId` para generar identificadores consecutivos.
- El método `addTask()` crea y registra nuevas tareas dentro de `this.tasks`.
- Cada tarea almacenada tiene un id único y un estado inicial `PORHACER`.
- El formulario Nueva Tarea utiliza `TaskManager` para registrar las tareas enviadas por el usuario.
- Después de registrar una tarea, el formulario queda limpio y listo para una nueva entrada.

SPRINT 2 · TAREA 5**DEL OBJETO AISLADO AL REGISTRO DE TAREAS**

En la Tarea 4 creaste la clase TaskManager y preparaste la propiedad tasks como una colección vacía. Ahora esa colección comenzará a recibir información real.

CONCEPTO CLAVE

TaskManager será el encargado de mantener el registro de las tareas. Cada vez que se cree una nueva tarea, addTask() incrementará el identificador y agregará un objeto al array tasks.

¿Qué cambia en esta tarea?

ANTES	AHORA	RESULTADO
tasks = []	tasks recibe nuevos objetos	Existe un registro de tareas
Sin identificador	currentId aumenta en cada ingreso	Cada tarea tiene un id único
Formulario independiente	submit llama a addTask()	El formulario alimenta TaskManager

Estructura que debes conservar

```
Planificador-Tareas/  
├── css/  
│   └── styles.css  
├── js/  
│   ├── index.js  
│   └── taskManager.js  
└── index.html
```

RECUERDA

Continúa trabajando sobre el mismo proyecto. No crees un proyecto nuevo para esta tarea. Conserva la estructura, el repositorio y el historial de commits construidos durante los sprints anteriores.

DESARROLLO DE LA ACTIVIDAD

PASO 1 · AÑADIR UNA NUEVA TAREA DE MANERA PROGRAMÁTICA

En este paso modificarás TaskManager para que pueda generar identificadores y registrar nuevas tareas dentro de `this.tasks`.

1. Actualizar el constructor

El constructor debe aceptar `currentId` como parámetro y utilizar 0 como valor inicial. Después, guarda ese valor en `this.currentId`.

```
class TaskManager {  
  constructor(currentId = 0) {  
    this.tasks = [];  
    this.currentId = currentId;  
  }  
}
```

2. Crear el método `addTask()`

Crea un método llamado `addTask`. Debe recibir la información necesaria para construir una nueva tarea:

- `name`
- `description`
- `dueDate`
- `status`

3. Incrementar el identificador

Cada vez que se ejecute `addTask()`, incrementa `this.currentId` antes de crear el nuevo objeto.

```
this.currentId++;
```

4. Registrar la tarea en `this.tasks`

Utiliza `Array.prototype.push()` para agregar un nuevo objeto al array. La tarea debe incluir el id generado y el estado correspondiente. Como regla de esta actividad, el estado inicial será `PORHACER`.

```
this.tasks.push({  
  id: this.currentId,  
  name: name,  
  description: description,  
  dueDate: dueDate,  
  status: 'PORHACER'  
});
```

¿POR QUÉ USAMOS `currentId`?

El array puede contener muchas tareas. El contador permite asignar 1, 2, 3, 4... sin repetir identificadores mientras la instancia de `TaskManager` siga registrando tareas.

PASO 1 - CONTINUACIÓN

EJEMPLO DE RESULTADO

Si agregas dos tareas, cada una debe quedar registrada como un objeto independiente con un identificador diferente.

```
const task1 = {
  id: 1,
  name: 'Sacar la basura',
  description: 'Sacar la basura al frente de la casa',
  dueDate: '2020-09-20',
  status: 'PORHACER'
};

const task2 = {
  id: 2,
  name: 'Hacer de cenar',
  description: 'Preparar una ración saludable para la familia',
  dueDate: '2020-09-20',
  status: 'PORHACER'
};
```

Prueba tu método antes de conectarlo al formulario

En js/index.js, crea una instancia de TaskManager, agrega una tarea utilizando addTask() y muestra la propiedad tasks en la consola.

```
const taskManager = new TaskManager();

taskManager.addTask(
  'Sacar la basura',
  'Sacar la basura al frente de la casa',
  '2020-09-20',
  'PORHACER'
);

console.log(taskManager.tasks);
```

RESULTADO ESPERADO

En la consola del navegador deberías observar un array con una tarea registrada. La tarea debe contener un id, sus datos y el estado PORHACER.

Prueba adicional recomendada

Agrega una segunda tarea antes del console.log(). Comprueba que su id sea diferente al de la primera.

```
taskManager.addTask(
  'Hacer de cenar',
  'Preparar la cena para la familia',
  '2020-09-20',
  'PORHACER'
);

console.log(taskManager.tasks);
```

DESARROLLO DE LA ACTIVIDAD

PASO 2 · AÑADIR TAREAS CON EL FORMULARIO

Ahora conectarás la interfaz con la lógica de TaskManager. El objetivo es que el envío del formulario desencadene la creación de una tarea y que los datos queden almacenados en tasks.

1. Inicializar TaskManager

Cerca de la parte superior de js/index.js, asegúrate de tener una instancia de TaskManager disponible.

```
const taskManager = new TaskManager();
```

2. Escuchar el evento submit

Selecciona el formulario Nueva Tarea con document.querySelector() y agrega un escuchador de eventos para detectar el envío.

```
const newTaskForm = document.querySelector('#newTaskForm');

newTaskForm.addEventListener('submit', function(event) {
  event.preventDefault();

  // aquí irá la lógica para registrar la tarea
});
```

IMPORTANTE

Debes prevenir la acción predeterminada del formulario utilizando event.preventDefault(). De esta manera, el navegador no recargará la página antes de que JavaScript procese la información.

3. Recuperar los valores de las entradas

Dentro del evento submit, utiliza los elementos del formulario para obtener los valores ingresados por el usuario. Si ya tienes lógica de validación, intégrala con el flujo existente en lugar de duplicarla.

4. Llamar a addTask()

Cuando la validación sea exitosa, pasa los valores obtenidos del formulario al método addTask() de taskManager.

```
taskManager.addTask(
  name,
  description,
  dueDate,
  status
);
```

5. Limpiar el formulario

Después de registrar correctamente la tarea, limpia los valores de las entradas para que el formulario quede preparado para una nueva tarea.

```
newTaskForm.reset();
```

COMPRENSIÓN

DEL CLIC EN “AGREGAR” AL ARRAY `tasks`

Piensa en la interacción como una cadena de responsabilidades. El formulario captura información; la validación determina si puede continuar; TaskManager crea y registra el objeto.

1 · USUARIO	2 · FORMULARIO	3 · JAVASCRIPT	4 · TASKMANAGER
Ingresa datos	Dispara submit	Valida y obtiene values	<code>addTask(...)</code>
Presiona guardar	Evita recarga	Llama al método	incrementa <code>currentId</code>
—	—	Limpia formulario	<code>push()</code> → <code>tasks</code>

Criterios funcionales

- El envío del formulario no recarga la página.
- Los datos ingresados se recuperan correctamente desde los campos.
- La validación existente continúa funcionando.
- Solo después de una validación exitosa se ejecuta `addTask()`.
- Cada nueva tarea recibe un id consecutivo.
- Las tareas quedan almacenadas dentro de `taskManager.tasks`.
- El formulario queda limpio después de registrar la tarea.

NO ADELANTES FUNCIONALIDADES

En esta tarea no necesitas implementar la eliminación de tareas ni persistencia en `localStorage`. El objetivo es crear y registrar tareas en memoria mediante TaskManager.

PRUEBA Y COMPRENSIÓN

CASOS DE PRUEBA

No te limites a probar únicamente el caso exitoso. Verifica también qué ocurre cuando el usuario envía información incompleta.

CASO	ACCIÓN	RESULTADO ESPERADO
1 · Formulario válido	Completa todos los campos y envía.	La tarea se registra en tasks y el formulario se limpia.
2 · Formulario vacío	Envía sin completar información.	La validación impide registrar una tarea.
3 · Campos incompletos	Completa solo algunos campos.	Se muestran los errores correspondientes y no se registra la tarea.
4 · Dos tareas	Registra dos tareas consecutivas.	Cada tarea tiene un id diferente.
5 · Consola	Inspecciona taskManager.tasks.	El array contiene las tareas creadas correctamente.

Si aparece un error

- Revisa que taskManager sea una instancia de TaskManager.
- Verifica que taskManager.js se cargue antes de index.js.
- Comprueba que el formulario tenga el id utilizado en querySelector().
- Revisa que los id de los inputs coincidan exactamente con los selectores usados en JavaScript.
- Comprueba que event.preventDefault() esté dentro del evento submit.
- Usa console.log() para verificar los valores antes de llamar a addTask().
- Verifica que currentId aumente y que this.tasks cambie después de cada registro.

Preguntas de comprensión

- ¿Por qué necesitamos un currentId además del array tasks?
- ¿Qué sucede con this.currentId cuando se agrega una segunda tarea?
- ¿Qué diferencia existe entre recuperar un valor con .value y almacenar una tarea en this.tasks?
- ¿Por qué es importante prevenir el comportamiento predeterminado del formulario?

CIERRE · TAREA 5

VERIFICACIÓN Y ENTREGABLE

Antes de finalizar, utiliza este checklist para comprobar que la funcionalidad está completa y que el repositorio refleja el avance de la tarea.

TaskManager

- ☐ El constructor acepta currentId con valor predeterminado 0.
- ☐ Existe la propiedad this.currentId.
- ☐ Existe el método addTask().
- ☐ addTask() recibe la información necesaria para crear una tarea.
- ☐ this.currentId se incrementa al agregar una tarea.
- ☐ La tarea se agrega mediante push() a this.tasks.
- ☐ Cada tarea incluye un id.
- ☐ El estado inicial se registra como PORHACER.

Formulario

- ☐ Existe una instancia de TaskManager en index.js.
- ☐ El formulario escucha el evento submit.
- ☐ Se utiliza event.preventDefault().
- ☐ La validación existente continúa funcionando.
- ☐ Los valores de los campos se recuperan correctamente.
- ☐ Los datos válidos llegan a addTask().
- ☐ El formulario se limpia después de registrar la tarea.

Pruebas

- ☐ Se registró al menos una tarea desde el formulario.
- ☐ Se registraron dos tareas y se comprobó que los id son diferentes.
- ☐ La consola muestra correctamente taskManager.tasks.
- ☐ El formulario no recarga la página al enviarse.
- ☐ Los datos inválidos no generan una tarea.

Proyecto y GitHub

- ☐ Los cambios fueron guardados en el proyecto existente.
- ☐ El repositorio contiene los avances de las tareas anteriores.
- ☐ Se realizó un commit que evidencie el avance de la Tarea 5.
- ☐ La estructura de archivos se mantiene organizada.

ENTREGABLE

Conserva tu proyecto actualizado en el repositorio de GitHub. La revisión considerará tanto la funcionalidad como la evolución del proyecto y la trazabilidad de tu trabajo.